

LEVELS 25-27

Chapter 25: Study Plans

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 25: Study Plans — 16-Week, Daily Routines, and Spaced Revision

← [Chapter 24: Resource Library](#) · [Index](#) · Next → [Chapter 26: Cheat Sheet & Decision Matrices](#)

25.1 The 16-Week Plan (≈15 hours/week, ≈240 hours total)

Each week below maps directly to this roadmap's chapters — follow the links as you go.

Week	Focus	This week's chapters	Practice	Weekly output
1	Networking + fundamentals	Ch.1 Prerequisites , Ch.2 Fundamentals	Watch ByteByteGo's HTTP/TCP/DNS videos alongside reading; do Chapter 1 & 2's interview drills out loud	Can explain CAP theorem and TCP vs UDP from memory, unscripted
2	Estimation + building blocks	Ch.3 Capacity Estimation , Ch.4 Core Building Blocks	Redo 3 of Chapter 3's worked examples <i>without</i> looking, then check yourself; drill Chapter 4's cache-failure-mode table from memory	Can do back-of-envelope math for an unfamiliar product in under 5 minutes
3	Databases	Ch.5 Database Deep Dive	Read DDIA Ch.5-6 (Replication, Partitioning) alongside; explain consistent hashing out loud, twice, until it's fluent	Can recite the full DB scaling ladder from memory
4	Distributed systems (the deep one — budget extra time)	Ch.6 Distributed Systems	Read DDIA Ch.7-9; do the full Chapter 6 interview drill; explain the Saga pattern out loud with a concrete example	Comfortable with idempotency, quorum math, and Saga vs. 2PC
5	Messaging + microservices	Ch.7 Messaging , Ch.8 Microservices	Given your Kafka background, focus study time on <i>articulating</i> what you already operate — record yourself explaining consumer group rebalancing	Can explain a cascading failure scenario and its 3 mitigations without notes

Week	Focus	This week's chapters	Practice	Weekly output
6	APIs + security	Ch.9 API Design , Ch. 10 Security	Critique 3 real public APIs you use daily against Chapter 9's good/bad checklist	Can design a clean, versioned, idempotent API surface live
7	Observability + cloud	Ch.11 Observability , Ch.12 Cloud AWS	Map your own Prometheus/Grafana/EKS setup onto Chapter 12's reference architecture — note the gaps	Can draw the full AWS reference architecture from memory
8	Patterns + interview framework + diagramming	Ch.13 Patterns , Ch. 14 Framework , Ch.15 Diagrams	Memorize the 10-step framework cold; practice drawing all 4 flow types from Chapter 15 by hand, timed	Can run the full 10-step framework on a random prompt in 45 min, alone, timed
9	Beginner + start intermediate problems	Ch.16 Beginner Problems , start Ch. 17 Intermediate	Do 3 beginner problems solo (45 min each, timed) before reading the chapter's answer	First self-timed 45-minute mocks, self-graded against Chapter 31's rubric
10	Finish intermediate problems	Ch.17 Intermediate (Twitter through Ticket Booking)	Do 4 intermediate problems solo, timed; start Mocks 1-5 (beginner tier) with a study partner if available	Comfortable with fan-out, feed, and inventory-race-condition problem shapes
11	Advanced problems, part 1	Ch.18 Problems 17-23 (Uber, Careem, Amazon, Netflix, Payment, Wallet, Ride Matching)	Mocks 6-10 (intermediate tier)	Fintech and marketplace problems feel like home territory
12	Advanced problems, part 2 + company prep	Ch.18 Problems 24-30 , Ch.19 Company Prep	Mocks 11-15 (senior tier)	Can name the specific reported question patterns for your top 3 target companies
13	FinTech + real-time + search + performance	Ch.20 , Ch.21 , Ch.22 , Ch.23	Redo the Payment System and Wallet problems from Chapter 18 from scratch, unaided	Double-entry ledger reasoning is fully automatic
14	Mock interviews, batch 1 + question bank	Ch.27 Question Bank , Mocks 1-10 review	3-4 full 45-min mocks this week, ideally with a partner playing interviewer	Self-score consistently 60+ on Chapter 31's rubric
15			3-4 more mocks, specifically the Tier-1-	Self-score consistently 75+

Week	Focus	This week's chapters	Practice	Weekly output
	Mock interviews, batch 2 + cheat sheet	Mocks 16–20 (Tier-1 level) , Ch.26 Cheat Sheet	tagged ones; identify your 3 weakest recurring gaps and re-read those specific sub-sections	
16	Final revision + company-specific sprint	Ch.19 , Ch.26 Cheat Sheet , spaced-revision pass (25.3 below)	Light — 1–2 mocks max, mostly review; re-read the cheat sheet daily; do the readiness checklist	Confident, not cramming — taper intensity going into real interviews

How to adapt this if you have less than 16 weeks: compress Weeks 1–8 (the learning phase) by dropping the "watch alongside" video time and reading this roadmap's chapters as your primary source, using YouTube only for topics that still feel shaky after reading — this roadmap's chapters are written to be sufficient on their own. Do not compress Weeks 9–16 (the practice phase) — mock-interview repetition volume is the least compressible, highest-ROI part of this whole plan.

25.2 Daily Routines

Pick the routine matching your actual available time — consistency beats intensity; a reliable 2 hours/day beats an inconsistent 4.

2-hour/day plan

Time	Activity
40 min	Learn — read one sub-section of the current week's chapter
30 min	Watch — one matched YouTube video reinforcing what you just read
30 min	Design — sketch one small piece of a problem (not a full 45-min mock every day — that's reserved for 2–3x/week)
20 min	Revise — redo yesterday's interview drill from memory, no notes

3-hour/day plan

Time	Activity
	Learn — read + take notes on the day's sub-topic

Time	Activity
45 min	
30 min	Watch — matched video content
20 min	Read — a relevant DDIA/System Design Interview book chapter section
60 min	Design — either a full 45-min timed mock (2-3x/week) or a focused deep-dive on one component of a problem (other days)
25 min	Revise — interview drill + explain one concept out loud, recorded if possible

4-hour/day plan

Time	Activity
60 min	Learn — full sub-topic, with notes
30 min	Watch — matched video
30 min	Read — book chapter section
90 min	Design — a full 45-min timed mock interview + 45 min self-review against the rubric
30 min	Revise — spaced-revision pass (25.3) + one interview drill

The repeatable core, regardless of which plan you pick: Learn → Watch → Read → Design → Revise, every single day, in that order. The "Design" step is the one people skip when short on time — don't; it's the highest-value 25–50% of your daily time, not the first thing to cut.

25.3 Spaced Revision System

Forgetting is the default; spaced revision is the deliberate counter to it. For every major topic (roughly one per chapter, or one per problem once you're in the problems phase), revisit it on this schedule:

Interval	What to do
Same day	Do the chapter's interview drill immediately after finishing the reading
Day 2	Re-answer the interview drill questions from memory, no notes — note anything you got wrong or fuzzy

Interval	What to do
Day 7	Re-answer again; if a specific question is still shaky, re-read just that sub-section (not the whole chapter)
Day 14	Explain the topic out loud, unprompted, as if teaching someone — this is a stronger test than answering a question, since nobody's prompting you with what to cover
Day 30	One final check — could you use this concept correctly inside a live 45-minute mock, under pressure, without it being the thing you're actively studying that day?

Practical implementation: keep a simple running list (a spreadsheet, a note, whatever's frictionless) of topics with the date you last touched them — when a topic crosses its next interval, it's due for a revisit. This doesn't need to be sophisticated software; the discipline of tracking it at all is what matters.

Revision checklist (run this weekly during Weeks 9-16 especially)

- [] Can I state the CAP theorem and PACELC precisely, including the partition-specific nuance?
- [] Can I recite the database scaling ladder in order, with the signal that triggers each step?
- [] Can I explain idempotency and walk through the idempotency-key pattern for a payment retry?
- [] Can I name all three cache failure modes and their fixes, unprompted?
- [] Can I explain the Saga pattern with a concrete 3-service example, including compensation?
- [] Can I run the full 10-step interview framework on a cold, unfamiliar prompt, alone, in 45 minutes?
- [] Can I draw all four diagram flow types (request, data, async, failure) cleanly, by hand?
- [] Can I explain double-entry ledger accounting with a worked example?
- [] Can I name my 3 weakest topics right now, honestly — and have I scheduled time to address them this week?

Next → [Chapter 26: Cheat Sheet & Decision Matrices](#) — the compact, pre-interview revision sheet and every requested architecture decision matrix in one place.